

Variables categóricas con muchos valores (alta cardinalidad)

Informe complementario a la clase 2.12 — Variables categóricas. Contexto: regresión lineal implementada con NumPy (ecuación normal + regularización), dataset de precios de autos.

1. El problema

La **cardinalidad** de una variable categórica es su número de valores distintos (`df[c].nunique()`). One-hot encoding crea **una columna por valor**, así que:

Variable (dataset de autos)	Valores distintos (aprox.)	Columnas con one-hot
<code>driven_wheels</code>	4	4
<code>vehicle_style</code>	16	16
<code>make</code>	48	48
<code>market_category</code>	~70 combinaciones	~70
<code>model</code>	~900	~900

Con muchas categorías aparecen cuatro problemas, todos importantes para la regresión lineal:

- 1 **Demasiadas columnas.** Con ~7.000 autos en train, agregar 900 columnas para `model` deja muy pocos ejemplos por peso `w`: el modelo memoriza (overfitting).
- 2 **Categorías raras.** Un valor que aparece 1 o 2 veces genera una columna casi toda en cero. Su peso se estima con 1–2 filas, así que es puro ruido.
- 3 **Matriz $X^T X$ singular o casi singular.** Columnas raras, columnas que son combinación de otras (p. ej. la suma de todas las dummies de una variable = 1 = columna del sesgo) hacen que $X^T X$ no tenga inversa estable. Es exactamente lo que hizo explotar el RMSE a ~41 en la clase 2.12.
- 4 **Categorías nuevas.** En validación/test (o en producción) aparecen valores que no estaban en train. El encoding tiene que saber qué hacer con ellos.

La guía propone quedarse con los **5 valores más frecuentes** y agrupar el resto en «otros». Es un buen punto de partida, pero el 5 es arbitrario y hay técnicas que aprovechan mejor la información. Este informe las ordena.

2. Primero, diagnosticar

Antes de elegir técnica, mirar **cómo se distribuyen** los valores. Casi siempre hay una *cola larga*: pocos valores concentran la mayoría de las filas.

```
for c in categorical_variables:
    vc = df_train[c].value_counts(dropna=False)
    cobertura = vc.cumsum() / vc.sum()
    print(c, 'distintos:', len(vc),
          '| top 5 cubren:', round(cobertura.iloc[min(4, len(vc) - 1)], 2),
          '| valores con < 10 filas:', (vc < 10).sum())
```

Preguntas a responder:

- ¿Cuántos valores distintos hay? ¿Cuántas filas tiene cada uno?
- ¿Qué porcentaje de filas cubren los N más comunes?
- ¿Hay valores que en realidad son lo mismo (*BMW*, *bmw*, *b.m.w.*)?
- ¿La variable es un **identificador** (casi un valor por fila)? Entonces no sirve.
- ¿Una celda contiene **varias categorías** a la vez ("*Crossover, Luxury*")?
- ¿Hay *NaN*? Un faltante puede ser información: tratarlo como una categoría más.

3. Técnicas

3.1 Limpiar y normalizar (siempre)

Antes de cualquier encoding: pasar a minúsculas, quitar espacios, unificar sinónimos y errores de tipeo. Ya se hizo en la clase 2.2 (`str.lower()`, `str.replace(' ', '_')`). Esto solo baja la cardinalidad sin perder nada.

3.2 Top-N + «otros», pero eligiendo N con criterio

Es la técnica de la guía. Dos formas mejores que fijar $N = 5$:

- **Por frecuencia mínima:** se queda con los valores que tienen al menos `min_count` filas en train (p. ej. 20–50). Así cada peso se estima con datos suficientes.
- **Por cobertura:** los valores más frecuentes hasta cubrir, p. ej., el 90 % de las filas.

Todo lo demás (y cualquier valor nuevo en val/test) va a la columna *otros*.

```
def categorias_frecuentes(serie, min_count=30):
    vc = serie.value_counts()
    return list(vc[vc >= min_count].index)

categories = {c: categorias_frecuentes(df_train[c]) for c in categorical_variables}
```

- **Ventajas:** simple, interpretable, controla el número de columnas.
- **Desventaja:** toda la información de las categorías raras se mezcla en una sola.

En scikit-learn: `OneHotEncoder(min_frequency=30, handle_unknown='infrequent_if_exist')` o `max_categories=10` hacen exactamente esto.

3.3 Agrupar con conocimiento del dominio (jerarquías)

En lugar de agrupar las raras en «otros» sin más, agruparlas en **algo que tenga sentido**. Es la técnica más potente cuando se conoce el negocio:

- *make* (48 marcas) → **origen** (alemana, japonesa, americana...) o **segmento** (lujo, generalista, deportiva). Pasa de 48 a ~5 columnas que generalizan bien: una marca de lujo rara hereda lo aprendido de las otras de lujo.
- *model* (~900) → su *make*, o su segmento. Un modelo raro deja de ser ruido.
- Códigos postales → comuna → región. Productos → subcategoría → categoría.

```
segmento = {'bmw': 'lujo', 'mercedes-benz': 'lujo', 'audi': 'lujo',
            'ferrari': 'super', 'lamborghini': 'super',
            'toyota': 'generalista', 'chevrolet': 'generalista'} # etc.
df['segmento'] = df.make.map(segmento).fillna('otro')
```

Luego se aplica one-hot a la nueva columna, que tiene pocos valores.

3.4 Frequency / count encoding

Reemplazar cada categoría por **cuántas veces aparece** en train (o su proporción). Una sola columna numérica, sin importar la cardinalidad.

```
freq = df_train.make.value_counts(normalize=True)
df['make_freq'] = df.make.map(freq).fillna(0) # valores nuevos → 0
```

Útil cuando la popularidad se relaciona con el target (en autos: las marcas masivas suelen ser más baratas). Problema: dos categorías con la misma frecuencia quedan iguales para el modelo.

3.5 Target encoding (mean encoding) con suavizado

Reemplazar cada categoría por el **promedio del target** en esa categoría. Para nuestro problema: cada **model** se reemplaza por el promedio de **log1p(msrp)** de los autos de ese modelo en train. Una sola columna, y muy informativa.

El riesgo es que una categoría con 2 filas tenga un promedio extremo. Por eso se **suaviza** hacia el promedio global, con un parámetro **m** (cuántas filas «imaginarias» con el promedio global se suman):

$$\text{encoding}(c) = (n_c \cdot \text{media}_c + m \cdot \text{media_global}) / (n_c + m)$$

- Si **n_c** es grande, manda la media de la categoría.
- Si **n_c** es chico, el valor se acerca a la media global.
- Una categoría nueva (**n_c = 0**) recibe exactamente la media global.

```
def target_encoding(train_col, y_train, m=20):
    media_global = y_train.mean()
    stats = pd.DataFrame({'c': train_col.values, 'y': y_train}) \
        .groupby('c')['y'].agg(['mean', 'count'])
    enc = (stats['count'] * stats['mean'] + m * media_global) / (stats['count'] + m)
    return enc, media_global

enc, media_global = target_encoding(df_train.model, y_train)
df_val['model_te'] = df_val.model.map(enc).fillna(media_global)
```

Cuidado con la fuga de información (leakage). Si se calcula el encoding de train con las mismas filas de train, la fila «ve» su propio target y el modelo confía demasiado en esa columna: se ve genial en train y mal en validación. La solución es **out-of-fold**: dividir train en K partes y codificar cada parte con las estadísticas de las otras K-1.

```
def target_encoding_oof(train_col, y_train, k=5, m=20, seed=1): # y_train: array de NumPy
    idx = np.arange(len(train_col))
    np.random.seed(seed)
    np.random.shuffle(idx)
    resultado = np.zeros(len(train_col))
    for fold in np.array_split(idx, k):
        resto = np.setdiff1d(idx, fold)
        enc, media = target_encoding(train_col.iloc[resto], y_train[resto], m)
        resultado[fold] = train_col.iloc[fold].map(enc).fillna(media).values
    return resultado

df_train['model_te'] = target_encoding_oof(df_train.model, y_train)
```

Para val/test se usa el encoding calculado con **todo** train (código anterior). Nunca se usa el target de validación ni de test para construir el encoding.

En scikit-learn ≥ 1.3: **TargetEncoder** hace el suavizado y el out-of-fold automáticamente dentro de **fit_transform**.

- **Ventajas:** una columna, aprovecha todas las categorías, funciona con miles de valores.
- **Desventajas:** riesgo de leakage si se hace mal; menos interpretable que one-hot.

3.6 Variables multi-etiqueta: **market_category**

market_category tiene valores como "**crossover, luxury, performance**". Tratar cada combinación como una categoría da ~70 valores, muchos raros. Pero las **etiquetas individuales** son solo ~10. Mejor separar y crear una columna binaria por etiqueta (*multi-hot*):

```
etiquetas = df_train.market_category.str.get_dummies(sep=',')
# columnas: crossover, diesel, exotic, factory_tuner, flex_fuel, hatchback,
#           high-performance, hybrid, luxury, performance
```

Un auto «crossover,luxury» tiene 1 en ambas columnas: el modelo aprende un peso para «luxury» con todos los autos de lujo, no solo con los de esa combinación.

3.7 Hashing y binary encoding (cardinalidad muy alta)

Para miles o millones de valores (IDs de usuario, URLs, códigos):

- **Feature hashing:** una función hash manda cada valor a una de K columnas fijas (p. ej. K = 32). Acepta valores nuevos sin problema. Hay colisiones (dos valores en la misma columna) y se pierde interpretabilidad.
- **Binary encoding:** numera las categorías y escribe el número en binario: 1.000 categorías → 10 columnas.

En regresión lineal estas columnas no tienen un significado claro, así que se usan poco; son más comunes en modelos de árboles o con datos masivos.

3.8 Ordinal encoding: solo si hay orden real

Asignar 0, 1, 2... a las categorías solo tiene sentido si existe un orden natural (**compact < midsize < large** en **vehicle_size**). Con marcas o modelos, el número inventa un orden que no existe, y la regresión lineal lo tomaría en serio («toyota = 2 × honda»). Para categorías sin orden: no usarlo.

3.9 Embeddings

En redes neuronales, cada categoría se representa con un vector denso aprendido (p. ej. 8 números). Es la técnica de los sistemas de recomendación. Fuera de alcance para regresión lineal, pero vale saber que existe.

3.10 Eliminar la variable

Si la variable es casi un identificador (un valor distinto por fila), no tiene nada que generalizar: se elimina. Si tiene una jerarquía, se reemplaza por el nivel superior (3.3).

4. Complementos obligatorios en regresión lineal

Regularización

Es lo que arregló el modelo en la clase 2.13: sumar r a la diagonal de $X^T X$. Con muchas columnas one-hot, la regularización **encoge** los pesos de las categorías con pocos datos hacia 0, lo que permite usar un umbral **min_count** más bajo sin que el modelo explote. Siempre ajustar r en validación después de cambiar el encoding.

La trampa de las dummies

Si se crean columnas para **todos** los valores de una variable y además hay sesgo w_0 , las columnas suman 1 en cada fila = columna del sesgo $\rightarrow X^T X$ singular. Opciones:

- Dejar fuera una categoría de referencia (**drop_first=True** en `pd.get_dummies`).
- O usar regularización, que vuelve invertible la matriz.

Con top-N sin columna «otros», la categoría de referencia es implícitamente «todo lo demás» (todas las columnas en 0), así que no hay trampa.

Ajustar solo con train

Toda decisión del encoding (qué categorías se quedan, frecuencias, medias del target) se calcula **solo con train** y se aplica igual a val y test. Es la misma idea que **prepare_X**: una única función que transforma todo de la misma manera.

5. Guía de decisión

Cardinalidad	Técnica recomendada (regresión lineal)
Baja (< 10–15)	One-hot completo (con categoría de referencia o regularización)
Media (15–100)	Agrupar por dominio, o top-N por frecuencia mínima + «otros», + regularización
Alta (100–1.000+)	Target encoding suavizado out-of-fold, jerarquía, o frequency encoding
Muy alta / IDs	Jerarquía, target encoding, hashing; o eliminar la variable
Multi-etiqueta	Separar etiquetas y hacer multi-hot
Con orden natural	Ordinal encoding

Se pueden **combinar**: por ejemplo, para **model** usar a la vez su **make** agrupado (one-hot) y su target encoding (una columna numérica).

6. Propuesta para el dataset de autos

Variable	Propuesta
<code>make</code>	One-hot con <code>min_count</code> ≈ 30 + regularización, o agrupar por segmento
<code>model</code>	Target encoding suavizado out-of-fold sobre <code>log1p(msrp)</code>
<code>market_category</code>	Multi-hot por etiqueta (<code>str.get_dummies(sep=',')</code>)
<code>engine_fuel_type</code> , <code>vehicle_style</code>	One-hot con <code>min_count</code> (pocas columnas)
<code>transmission_type</code> , <code>driven_wheels</code>	One-hot completo
<code>vehicle_size</code>	Ordinal (<code>compact=0</code> , <code>midsize=1</code> , <code>large=2</code>) o one-hot
<code>number_of_doors</code>	One-hot (2, 3, 4)

Cómo validar cada decisión (proceso de selección de modelos del módulo 1):

- 1 Partir del modelo actual (base + `age` + categóricas top-5 + regularización).
- 2 Cambiar **una** cosa a la vez (p. ej. el encoding de `model`).
- 3 Reajustar `r` y comparar el **RMSE en validación**.
- 4 Quedarse con el cambio solo si mejora. Test se usa una sola vez, al final.

7. Checklist

- ☐ Normalicé los textos (minúsculas, espacios, sinónimos).
- ☐ Miré `nunique()` y `value_counts()` de cada categórica en train.
- ☐ Revisé si hay variables numéricas que son categóricas (`number_of_doors`).
- ☐ Revisé si hay variables multi-etiqueta (`market_category`).
- ☐ Elegí el umbral de categorías por frecuencia/cobertura, no un N fijo.
- ☐ Pensé en agrupaciones de dominio (jerarquías).
- ☐ Si usé target encoding: suavizado + out-of-fold + solo target de train.
- ☐ Valores nuevos y `NaN` en val/test tienen un destino definido.
- ☐ No caí en la trampa de las dummies (o hay regularización).
- ☐ Reajusté `r` y comparé RMSE en validación.

Volver a: [2.12](#) — Variables categóricas · Índice del módulo